Formalizing FIRRTL Intermediate Representation and Its Lowering Transformations

ANONYMOUS AUTHOR(S)
SUBMISSION ID: #792

FIRRTL is an intermediate representation for the Chisel hardware construction language and has been intensively used in designing RISC-V CPUs. FIRRTL features high-level constructs like vector and bundle types, which are then removed by a sequence of lowering transformations, resulting in a most restricted form, called low FIRRTL. This paper presents Vefirrtl, a framework to formally reason about programs in FIRRTL and its lowering transformations. Vefirrtl provides a mechanized formal semantics for low FIRRTL, and formalizes the syntax of FIRRTL and its lowering transformations, all in the interactive theorem prover Coq. To validate the formal semantics for low FIRRTL, we extract from it an OCaml interpreter that can execute low FIRRTL programs, and empirically check its consistency against Treadle, the low FIRRTL reference interpreter, on a benchmark suite comprising 120 low FIRRTL programs. Experimental results show that our OCaml interpreter is 6.95 times faster than Treadle on average, and produces the same outputs as Treadle for almost all the benchmarks except for one due to an undefined semantics in low FIRRTL. Furthermore, we specify and verify the functional correctness of two key lowering transformations, namely "ExpandConnects" and "ExpandWhens", which expands the connect statements for vector and bundle types, and replaces conditional statements with multiplexers, respectively.

CCS Concepts: • **Hardware → Hardware description languages and compilation**; **Software tools for EDA**; • **Theory of computation → Logic and verification**; **Semantics and reasoning**.

Additional Key Words and Phrases: RISC-V CPU design, FIRRTL, lowering transformations, formal semantics, verification

ACM Reference Format:
Anonymous Author(s). 2018. Formalizing FIRRTL Intermediate Representation and Its Lowering Transformations. *Proc. ACM Program. Lang.* 1, CONF, Article 1 (January 2018), 29 pages.

1 INTRODUCTION

Compilers are among the most fundamental and widely-used software, based on which programs in high-level programming languages are translated into low-level code. Well-known open source compilers include GCC [?] and LLVM [?] for compiling C and C++ programs. Compilers for mainstream programming languages like C usually include many compilation passes and optimizations. As a result of their inherent complexity, there remain massive bugs in compilers, despite intensive testing efforts [??]. Even worse, compiler bugs could silently entail incorrect executions in low-level programs and lead to catastrophic consequences for safety-critical applications, and it is extremely difficult to determine whether an execution failure is caused by defects in the programs or bugs in compilers. This motivates researchers to apply formal verification techniques to guarantee the correctness of compilers.

Formally verified compilers have been built for several realistic programming languages, such as CompCert for C [??????], Vélus for Lustre [?], Vellvm [??] and Crellvm [?] for the LLVM Intermediate Representation, and CakeML for ML [????]. Such compilers come with a mathematical, machine-checked proof that compilation passes preserve program semantics, hence strengthening the correctness guarantees that can be offered by formal verification of high-level programs to low-level programs.

2018. 2475-1421/2018/1-ART1 $15.00
https://doi.org/

The success of formally verified compilers on software programming languages has been ported to hardware description languages (HDLs) very recently, e.g., the extension of CakeML (called Lutsig) for compiling a subset of register-transfer level (RTL) Verilog programs to netlist level programs for field-programmable gate array (FPGA) [**??**], and the extension of CompCert (called Vericert) for compiling a subset of C programs to RTL Verilog programs [**?**]. Following this promising research direction, in this work, we establish for the first time a framework that formally reasons about programs in FIRRTL [**?**] (Flexible Intermediate Representation for RTL) and its lowering transformations (aka compilation passes).

FIRRTL originated from the high-level hardware construction language Chisel [**?**]. Chisel is a domain specific language embedded in Scala that comprises a library of special class definitions, predefined objects, and usage conventions for writing highly-parameterized hardware design generators. High-level hardware generators in Chisel are developed by manipulating circuit components using Scala functions, and their interfaces are encoded by Scala types. Hardware can also be constructed by leveraging Scala's functional and object-oriented programming features, thus enabling parameterized, modular, and reusable hardware generators that improve the productivity and robustness of hardware designs. Chisel has been successfully used to build RISC-V processes such as Rocket Chip [**?**], RISC-V BOOM [**?**], NutShell [**?**], XiangShan [**??**], and so on.

FIRRTL as an intermediate representation (IR) was proposed for the compilation of Chisel programs [**?**], analogous to the role of LLVM IR [**?**] for compiling C programs. FIRRTL features first-class support for high-level constructs such as vector types, bundle types, conditional statements, connects, partial connects, and modules so that Chisel programs can easily be translated into FIRRTL. High-level synthesis is performed on FIRRTL by a sequence of compilation passes and optimizations, resulting in a most restricted form that is close to RTL Verilog, called low FIRRTL (abbreviated as LoFIRRTL). For clarity, let us call FIRRTL with high-level constructs *high FIRRTL* (abbreviated as HiFIRRTL). While FIRRTL was designed to compile Chisel programs, it is not tied to Chisel. Other hardware description languages can also be compiled into FIRRTL and reuse the majority of the FIRRTL compilation passes. For instance, FIRRTL has been integrated into the CIRCT project [**?**] as one of its core dialects.

Similar to C/C++ compilers (e.g., GCC and LLVM), the FIRRTL compiler also suffers from lots of bugs due to its inherent complexity. For instance, there are 655 issues in its Github repository, among which 99 have been labeled as bugs [**?**]. This motivates us to investigate the formalization and verification of FIRRTL and its lowering transformations. Nevertheless, partially due to the fact that Chisel and FIRRTL are relatively new and the ecosystem around them is still evolving, formalization and verification of FIRRTL and its lowering transformations are largely missing, with only an exception, i.e. [**?**]. In [**?**], a coinductive denotational semantics for an idealized subset of LoFIRRTL was proposed and formalized in the Agda proof assistant, based on a formalism called device calculus. Nevertheless, while [**?**] made some progress in the formalization of LoFIRRTL, a general framework where programs in HiFIRRTL and its lowering transformations can be formally reasoned about is still elusive. The establishment of such a framework would require a mechanized operational semantics of LoFIRRTL, validation of the conformance of the semantics to the FIRRTL reference interpreter, formalization of HiFIRRTL and its lowering transformations, as well as specification and verification of the correctness of its lowering transformations.

Contributions. In this work, we fill in this gap and establish a general framework called Vefirrtl for formally reasoning about programs in FIRRTL and its lowering transformations. Specifically, we make the following major contributions.

- Define an operational semantics for LoFIRRTL, mechanized in Coq. The operational semantics is based on a concept of pseudo-single-static-assignment (pseudo-SSA for short)

form of LoFIRRTL programs. Pseudo-SSA form enables us to define the operational semantics of LoFIRRTL in a sequentially compositional way, much like those for software. Our formalization covers generic features of LoFIRRTL for digital circuits, but with memories excluded, since their precise semantics is still unclear and under active debate. To avoid the potential inconsistencies, we choose to omit them in this work and focus on the others whose semantics can be made clear.

- We extract an OCaml interpreter for LoFIRRTL from the operational semantics and validate it against Treadle, the main-stream FIRRTL reference interpreter, on a benchmark suite comprising 120 LoFIRRTL programs. The benchmarks are collected from various sources, including the LoFIRRTL programs obtained from compiling the Chisel programs in the Chisel book and LoFIRRTL programs used for testing the Treadle interpreter. We analyze the branch coverage of these benchmarks for testing the OCaml interpreter and design more LoFIRRTL programs manually to cover the missing branches. The experiments show the consistency of the two interpreters, with only one exception, that is, the inconsistency between the official specification and Treadle implementation about the semantics of `validif` expressions.
- Introduce a concept of circuit diagrams and formalize the syntax of HiFIRRTL and its six lowering transformations in Coq, namely, "InferTypes", "InferWidths", "InferResets", "ExpandConnects", "ExpandWhens", "LowerTypes". In circuit diagrams, the information about types and connections of components are extracted and stored explicitly in two mappings, which makes it easier to formalize and verify the lowering transformations. The aforementioned six lowering transformations are believed to be the essential ones for compiling HiFIRRTL programs into LoFIRRTL ones.
- Specify and verify the functional correctness of the two most involved and impactful lowering transformations in Coq, namely "ExpandConnects" and "ExpandWhens", which make dramatic changes to HiFIRRTL programs, e.g. creating new connect statements.

The establishment of the Vefirrtl framework for reasoning about programs in FIRRTL and its lowering transformations lays down a solid foundation towards a fully verified compiler for FIRRTL.

Organization. The rest of this paper is organized as follows. Section 2 gives an informal introduction to FIRRTL and its lowering transformations. Section 3 is devoted to the operational semantics of LoFIRRTL as well as its validation against Treadle. Section 4 formalizes HiFIRRTL and its lowering transformations. Section 5 specifies and verifies "ExpandConnects" and "ExpandWhens". We discuss related work in Section 6 and conclude this paper in Section 7.

2 FIRRTL IR AND ITS TRANSFORMATIONS

In this section, we give an informal introduction to FIRRTL and its lowering transformations.

2.1 FIRRTL IR

A FIRRTL *circuit* comprises a list of modules with a designated top-level module that has the same identifier as the circuit. Each module represents a hardware block that can be instantiated one or more times. For instance, the FIRRTL circuit `A` shown in Figure 1 has two modules `A` and `B`, where the top-level module of the circuit is the module `A`. A *module* consists of a set of ports and a sequence of statements. Each port has an identifier, a direction (either `input` or `output`), and a data type. For instance, the module `A` has two input ports `clk` and `rst` and one output port `io`, with types `Clock`, `UInt<1>` and `{flip enable: UInt<1>, calc: UInt<3>}`, respectively. Each *statement* defines a wire, register, node, connect (`<=`), partial connect (`<-`), conditional (`when`), or instance (`inst of`),

```
circuit A :
  module A :
    input clk : Clock
    input rst : UInt<1>
    output io : {flip enable : UInt<1>, calc : UInt<3>}
    clk is invalid
    rst is invalid
    io is invalid
    inst x of B
    x.io is invalid
    x.clk <= clk
    x.rst <= rst
    x.io.enable <= io.enable
    io.calc <= x.io.calc

  module B :
    input clk : Clock
    input rst : UInt<1>
    output io : {flip enable : UInt<1>, calc : UInt<3>}
    clk is invalid
    rst is invalid
    io is invalid
    reg counter : UInt<3>, clock with : (reset => (rst, UInt<3>("h00")))
    when io.enable
      node _T_7 = add(counter, UInt<1>("h01"))
      node _T_8 = tail(_T_7, 1)
      counter <= _T_8
      skip
    io.calc <= counter
```

Fig. 1. An example FIRRTL program

and so on. A *type* for specifying the structure of the data can be either a *ground type,* e.g. `Clock` (carrying a clock signal) and `UInt<1>` (unsigned integer of bit width 1); or an *aggregate type*, which is either a *vector type,* e.g. `UInt[3]` (a three-element vector of unsigned integers), or a *bundle type,* e.g. `{flip enable: UInt<1>, calc: UInt<3>}` comprising two signals of the ground types `UInt<1>` and `UInt<3>` respectively. A circuit *component* is a declared element in a FIRRTL circuit that can be referred to by its name or identifier. Therefore, a component can be a port, node, wire, register, memory, or instance. A declared component can be connected to or from except for input ports that can only be connected from.

The syntactic rules of FIRRTL are given in Figure 2 in Extended Backus-Naur Form (EBNF), where $\{\cdots\}$ denotes repetition (repeating zero or more times) and $[\cdots]$ denotes option (either included or discarded), moreover, the terminals are doubly quoted by “ and ”. We stress that the syntactic rules of FIRRTL presented here is a simplification of those in the official specification[1]. For instance, indent, dedent and newline terminals are omitted for readability; moreover, the types and constructs for analog signals are also omitted, since we focus on digital circuits in this work. In the syntax, memory is also omitted since its semantics is still under active debate.[2]

[1] https://github.com/chipsalliance/firrtl-spec/blob/main/spec.md
[2] https://github.com/chipsalliance/chisel/issues/3112

```
c          ::= "circuit" id":"{m}
m          ::= "module" id{p}{s}
p          ::= ("input" | "output") id":"t
s          ::= "wire" id":"t | "reg" id":"t e["(with:{reset=>("e","e")})"]
               | "inst" id "of" id | "node" id"="id | r"<="e | r"<-"e | r "is invalid"
               | "when" e":"{s}["else:"{s}] | "skip"
e          ::= ("UInt"|"SInt")[w]"("int")" | r | "mux("e","e","e")"
               | "validif("e","e")" | "read("sr")" | op
r          ::= sr | r"["e"]"
sr         ::= id | sr"."id | sr"["int"]"
w          ::= "<"int">"
t          ::= tg | ta
tg         ::= "Clock" | "Reset" | "AsyncReset" | ("UInt" | "SInt")[w]
ta         ::= "{"f{f}"}" | t"["int"]"
f          ::= ["flip"] id":"t
id         ::= ("_" | l){"_" | l | d}
op         ::= op_2e | op_1e | op_1e1i | op_1e2i
op_2e      ::= op_2e_key"("e","e")"
op_2e_key  ::= "add" | "sub" | "mul" | "div" | "mod" | "lt" | "leq" | "gt" | "geq"
               | "eq" | "neq" | "dshl" | "dshr" | "and" | "or" | "xor" | "cat"
op_1e      ::= op_1e_key"("e")"
op_1e_key  ::= "asUInt" | "asSInt" | "asClock" | "cvt" | "neg" | "not" | "andr" | "orr" | "xorr"
op_1e1i    ::= op_1e1i_key"("e","int")"
op_1e1i_key ::= "pad" | "shl" | "shr" | "head" | "tail"
op_1e2i    ::= op_1e2i_key"("e","int","int")"
op_1e2i_key ::= "bits"
int        ::= ["-"]d{d} | ""b"["-"]d_b{d_b}""" | ""o"["-"]d_o{d_o}""" | ""h"["-"]d_h{d_h}"""
```

d (resp. d_b, d_o and d_h) denotes a decimal (resp. binary, octal and hexadecimal) digit, and l denotes a letter

Fig. 2. Syntax of FIRRTL

According to the syntax, FIRRTL features high-level constructs like vector types, bundle types, conditional statements, connects, partial connects, and modules. These high-level constructs are then gradually removed by a sequence of lowering transformations. During each lowering transformation, the circuit is rewritten into an equivalent circuit using simpler, lower-level constructs. Eventually the circuit is simplified to its most restricted form, resembling a structured netlist, which allows for easy translation to an output language (e.g. Verilog). This form is given the name lowered FIRRTL (abbreviated as LoFIRRTL) and is a strict subset of the full FIRRTL language. For clarity, the original form is called high FIRRTL (HiFIRRTL). Besides LoFIRRTL, there is another lowered form of FIRRTL, called MidFIRRTL, which resides between HiFIRRTL and LoFIRRTL. Nevertheless, in this work, we do not introduce MidFIRRTL, since it can be seen as an internal representation of a FIRRTL compiler that does not interface with the other parts of the Chisel toolchain.

A FIRRTL circuit is a LoFIRRTL circuit if it obeys the following restrictions:

- All widths are explicitly defined.
- No conditional statements (`when`) are used.
- All circuit components are declared with a ground type.
- No partial connect statements (<-) are used.
- Each component is connected to at most once.

More precisely, a LoFIRRTL circuit is a FIRRTL circuit that is defined by the syntactic rules in Figure 3 and satisfies that in each module, each component is connected to at most once. Note

$$
\begin{array}{lcl}
s & ::= & \text{“}\texttt{wire}\text{”}\ id\text{“:”}t \mid \text{“}\texttt{reg}\text{”}\ id\text{“:”}t\ e[\text{“}(\texttt{with:\{reset=>(}\text{”}e\text{“,”}e\text{“)\}})\text{”}] \\
 & & \mid \text{“}\texttt{inst}\text{”}\ id\ \text{“}\texttt{of}\text{”}\ id \mid \text{“}\texttt{node}\text{”}\ id\text{“=”}id \mid r\text{“<=”}e \mid r\ \text{“}\texttt{is invalid}\text{”} \mid \text{“}\texttt{skip}\text{”} \\
e & ::= & (\text{“}\texttt{UInt}\text{”}|\text{“}\texttt{SInt}\text{”})\ w\text{“(”}int\text{“)”} \mid r \mid \text{“}\texttt{mux(}\text{”}e\text{“,”}e\text{“,”}e\text{“)”} \\
 & & \mid \text{“}\texttt{validif(}\text{”}e\text{“,”}e\text{“)”} \mid \text{“}\texttt{read(}\text{”}sr\text{“)”} \mid op \\
r & ::= & sr \mid r\text{“[”}e\text{“]”} \\
sr & ::= & id \mid sr\text{“.”}id \mid sr\text{“[”}int\text{“]”} \\
w & ::= & \text{“<”}int\text{“>”} \\
t & ::= & tg \\
tg & ::= & \text{“}\texttt{Clock}\text{”} \mid \text{“}\texttt{AsyncReset}\text{”} \mid (\text{“}\texttt{UInt}\text{”} \mid \text{“}\texttt{SInt}\text{”})w
\end{array}
$$

Fig. 3. Syntax of LoFIRRTL (Rules that are identical to those in Figure 2 are not repeated)

that in Figure 3, the rules for f, id, op are the same as those in Figure 2, thus omitted to avoid tediousness. In a nutshell, the syntactic rules in Figure 3 are obtained from those in Figure 2 by replacing $[w]$ with w (so that the width becomes mandatory), removing the `when` statement and the partial connect statement (<-) from the rule for s, and removing the rule for aggregate types ta. The requirement that each component is connected to at most once gives LoFIRRTL a flavor similar to static single-assignment (SSA) form of software programs. Programs in SSA form are those where each variable is assigned at most once and each variable is defined before it is used. For instance, the program in Figure 4(a) is a LoFIRRTL program, while the program in Figure 4(b) is not, since the output port `myport1` is connected to twice from the input ports `a` and `b`, respectively.

```
module MyModule :
  input a: UInt<1>
  input b: UInt<1>
  output myport1: UInt<1>
  output myport2: UInt<1>
  myport1 <= b
  myport2 <= a
```

(a)

```
module MyModule :
  input a: UInt<1>
  input b: UInt<1>
  output myport1: UInt<1>
  output myport2: UInt<1>
  myport1 <= a
  myport1 <= b
  myport2 <= a
```

(b)

Fig. 4. Examples for illustrating the syntactic restrictions of LoFIRRTL

2.2 Lowering Transformations of FIRRTL

In the sequel, we give an informal introduction to the lowering transformations (aka compilation passes) of FIRRTL. The dependencies between these compilation passes are illustrated in Figure 5.

InferTypes. The InferTypes pass infers the types of components. For instance, InferTypes infers from the code snippet `wire x: UInt x <= UInt<5>(15) x <= UInt<1>(1)` that the type of the wire x is `UInt`. Note that for a connect statement `x <= e`, InferTypes only checks that the type of the expression e is equivalent to that of the component x (where the bit widths are ignored), and does not update the bit width of x to that of the expression e.

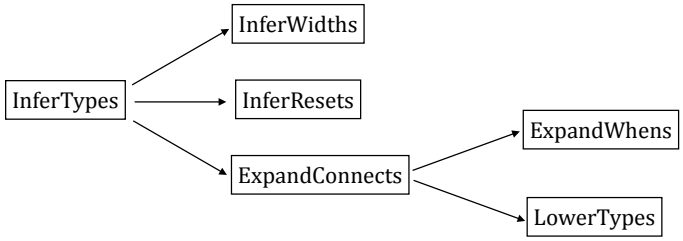

Fig. 5. Dependencies between FIRRTL compilation passes

InferWidths. The InferWidths pass infers the widths of all components whose widths are unknown, after the InferTypes pass has been applied. It is a *global* width inference algorithm. Instances of the same module with unknown input port widths will be assigned the largest width of all assignments to each of its instance ports. It infers the smallest width that is larger than all assigned widths to a component. Note that this means that dummy assignments that are overwritten by last-connect semantics can still influence width inference. The last-connect semantics means that the last connect statement for a component will take effects and all the preceding connect statements for it will be ignored in the end. For instance, InferWidths infers from the code snippet `wire x: UInt x <= UInt<5>(15) x <= UInt<1>(1)` that the width of the wire x is 5, but with a value `UInt<1>(1)`.

InferResets. The InferResets pass infers the concrete reset types for the abstract reset type `Reset`, namely, asynchronous resets represented by `AsyncReset` or synchronous resets represented by `UInt<1>`. For example, in the code snippet `input in: UInt<1> wire rst: Reset rst <= in` with an input port `in` of type `UInt<1>` and a wire `rst` of abstract type `Reset`, the `rst` wire will be inferred as a synchronous reset based on the type of the input port `in`. If a component of `Reset` type is instantiated with different concrete reset types, then a conflict occurs and an error is reported.

ExpandConnects. The ExpandConnects pass expands the connect and partial connect statements, after the types of components have been inferred by the InferTypes pass. For instance, the ExpandConnects pass transforms the program shown in Figure 6(a) into the one shown in Figure 6(b).

```
1 module MyModule:
2   input x: {a: UInt, b: UInt}
3   output y:{a: UInt, b: UInt}
4   y <= x
```

(a)

```
1 module MyModule:
2   input x: {a: UInt, b: UInt}
3   output y:{a: UInt, b: UInt}
4   y.a <= x.a
5   y.b <= x.b
```

(b)

Fig. 6. Examples for illustrating the ExpandConnects pass

ExpandWhens. The ExpandWhens pass mainly replaces `when` statements; this is necessary because a physical circuit cannot contain conditional elements. It also applies the so-called last-connect semantics.

For every ground-type writable (sub)component, ExpandWhens retains the last connect or `is invalid` statement that appears under any combination of `when` conditions in the module. If the

(sub)component is declared under multiple conditions, ExpandWhens selects between them using a multiplexer.

```
1  module TestCondition:
2    input in: UInt<32>
3    input condition: UInt<1>
4    output posv: UInt<32>
5    output negv: SInt<33>
6    posv <= 0
7    when condition:
8      posv <= in
9      negv is invalid
10   else:
11     wire a: SInt<33>
12     a <= neg in
13     negv <= a
```

(a)

```
1 module TestCondition:
2   input in: UInt<32>
3   input condition: UInt<1>
4   output posv: UInt<32>
5   output negv: SInt<33>
6   wire a: SInt<33>
7   posv <= mux(condition, in, 0)
8   a <= neg in
9   negv <= validif(not condition, a)
```

(b)

Fig. 7. Examples for illustrating the ExpandWhens pass

For instance, the ExpandWhens pass transforms the program shown in Figure 7(a) into the one shown in Figure 7(b). Note that according to the last connect semantics, the connect statement in Line 6 of Figure 7(a) is overridden by the one in Line 8 of Figure 7(a) if `condition` is true. Because of this overriding, ExpandWhens can only run after the connect statements have been expanded by ExpandConnects.

If, under some combination of conditions, the (sub)component is declared but is not connected to, the compiler assumes that the programmer has forgotten a connect statement and produces an error. This error can be suppressed by an `is invalid` statement (Line 9 of Figure 7(a)), which explicitly indicates that the connect is missing on purpose, e.g. because the result is not used or is irrelevant under some conditions. The multiplexer is replaced by a `validif` expression.

If a component is only declared under some condition (e.g. `wire a` in Line 11 of Figure 7(a))), it is nevertheless included in the LoFIRRTL result, and the condition is ignored; no multiplexer is inserted upon connections.

LowerTypes. The LowerTypes pass flattens bundles and vectors, after the connect and partial connect statements have been expanded by the ExpandConnects pass. Nevertheless, LowerTypes does not change the implicit bundles, e.g. module instances. After the application of the LowerTypes pass, a module instance is still a bundle of depth 1 (instance -> port).

3 FORMALIZATION OF LOW FIRRTL

3.1 Operational Semantics of LoFIRRTL

In the sequel, we introduce an operational semantics of LoFIRRTL programs, based on which LoFIRRTL is formalized in Coq. We first present the operational semantics for LoFIRRTL programs that are free of `inst of` statements and later consider the general LoFIRRTL programs.

3.1.1 `inst of`*-free LoFIRRTL.* Let us fix a LoFIRRTL circuit A that is free of `inst of` statements. According to the syntax of LoFIRRTL, the top-level module of the circuit must be module A.

We first introduce some notations required to present the operational semantics.

```
module MyModule :
  input a: UInt<1>
  input b: UInt<1>
  output myport1: UInt<1>
  output myport2: UInt<1>
  myport1 <= b
  myport2 <= a
```

(a) Example in pseudo-SSA form

```
module A
  input io_in: UInt<32>
  output io_out: UInt<32>
  io_out <= tail(a, 1)
  node a = add(io_in, UInt<32>("h1"))
```

(b) Example not in pseudo-SSA form

Fig. 8. Examples for pseudo-SSA form

A *generalized connect statement* is a statement of the form z <= e or node $z = e$, where z and e are called the *head* and *body* of the generalized connect statement, respectively. Intuitively, a generalized connect statement is either a connect statement z <= e or a node-declaration statement node $z = e$.

We denote by InPorts(A), OutPorts(A), Wires(A), Nodes(A) and Regs(A) the sets of input ports, output ports, wires, nodes and registers of the module A, respectively. Furthermore, we use Ports(A) to denote the set of ports InPorts(A) $\uplus$ OutPorts(A), Cpnts(A) to denote the set of components Ports(A) $\uplus$ Wires(A) $\uplus$ Nodes(A) $\uplus$ Regs(A), and CCpnts(A) to denote the set of components OutPorts(A) $\uplus$ Wires(A) $\uplus$ Nodes(A) that must be connected to, where a component z is connected to via a generalized connect statement whose head is z. Note that the input ports InPorts(A) are not connected to, moreover, the registers may not be connected to, thus they are not included in CCpnts(A). (If a register is not connected to, then it keeps the current value.) We denote by GTypes the set of ground types $(\{\texttt{UInt}, \texttt{SInt}\} \times \mathbb{N}) \cup \{\texttt{Clock}, \texttt{Reset}, \texttt{AsyncReset}\}$ and by BitVec the set of bit vectors $\{0, 1\}^+$. For each $t = (\tau, n) \in \{\texttt{UInt}, \texttt{SInt}\} \times \mathbb{N}$, we define the *width* of t, denoted by $wd(t)$, as n.

To characterize the dependency of components in the module A (hence define the semantics of the module A sequentially), we define a dependency graph of the module A. Formally, the *dependency graph* G_A of the module A is a directed graph (N_A, E_A), where N_A is the set Cpnts(A) of components, $E_A \subseteq$ Cpnts(A) $\times$ Cpnts(A) is a finite set of directed edges such that $(x, y) \in E_A$, i.e., y depends on x, iff A contains a generalized connect statement with head y and body e involving x.

To forbid illegal behaviors, we assume that the dependency graph G_A is *acyclic*, otherwise the module A has some combinational loops[3], which are usually taken as "bad" in digital circuit design. We develop a verified topological sorting algorithm for checking if a dependency graph G_A is cyclic or not. To have a well-defined semantics, we also assume that each component $z \in$ CCpnts(A) should be connected to exactly *once*, that is, there is exactly one generalized connect statement whose head is z. This assumption is checked by the "ExpandWhens" compilation pass and an error will be reported if the assumption is violated. Furthermore, the functional correctness of the "ExpandWhens" compilation pass is also proved in Coq. Finally, in this work, we focus on synchronization circuits, that is, there is one unique global clock.

To define the operational semantics of the module A, we require that it is in *pseudo-SSA form*, that is, the statement sequence of the module A satisfies the following two additional constraints,

- each component $z \in$ Cpnts(A) is declared before it is used in the bodies and heads of generalized connect statements (except that node $z \in$ Nodes(A) is declared and used simultaneously),

[3]A combinatorial loop is when the output of some combinatorial logic, that is a section of logic with no registers between gates, is fed back into the input of that combinatorial logic with no intervening register.

- the concatenation $\pi_1 \| \pi_2$ of a sequence π_1 of input ports and the sequence π_2 of heads of the generalized connect statements in the module A forms a topological sorting of the dependency graph G_A (note that input ports $\mathsf{InPorts(A)}$ do not have incoming edges in G_A, thus do not occur in the sequence π_2), specifically, for every edge (x, y) in the dependency graph G_A, the component x precedes the component y in the sequence $\pi_1 \| \pi_2$.

Intuitively, the first condition ensures that each component $z \in \mathsf{Cpnts(A)}$ has a declared data type before it is used in the bodies and heads of connect statements. The second condition ensures that a generalized connect statement with head x must precede any generalized connect statement with head y in the module A if (x, y) is an edge in the dependency graph G_A, namely, the signal of x is evaluated before it is used for evaluating y. For instance, the LoFIRRTL program shown in Figure 8a is in pseudo-SSA form while the one shown in Figure 8b is not, since $(\mathtt{a}, \mathtt{io_out})$ is an edge in G_A, while the generalized connect statement `node a = add(io_in, UInt<32>("h1"))` for the component `a` follows the generalized connect statement `io_out <= tail(a, 1)` for the component `io_out`. In the sequel, we assume that the module A is in the pseudo-SSA form.

The operational semantics of LoFIRRTL relies on a concept of configurations. A *configuration* of the module A is a triple (te, s, rs), where $te : \mathsf{Cpnts(A)} \to \mathsf{GTypes}$ is a type environment, $s : \mathsf{Cpnts(A)} \to \mathsf{BitVec} \cup \{\bot\}$ is a state, and $rs : \mathsf{Regs(A)} \to \mathsf{BitVec} \cup \{\bot\}$ is a register state, $\bot$ denotes undefined. Intuitively, te denotes the types of the components, s denotes the (signal) values of the components in the beginning of the current clock cycle, and rs denotes the new values of the registers in the beginning of the next clock cycle which will be committed to the next state at the arriving of the next clock signal. The type environment te and state s are lift to expressions e as usual, whose definitions can be found in the supplementary material.

The operational semantics of each statement st in the module A is defined as a transition relation between configurations

$$(te, s, rs) \xrightarrow{st} (te', s', rs'),$$

which denotes that the execution of the statement st from a configuration (te, s, rs) leads to a configuration (te', s', rs'). Moreover, the operational semantics of a sequence of statements is defined as the composition of the relations of statements therein and the operational semantics of module A is defined as the operational semantics of its statement sequence. For instance, we have $(te, s, rs) \xrightarrow{st_1\ st_2} (te'', s'', rs'')$, if $(te, s, rs) \xrightarrow{st_1} (te', s', rs')$ and $(te', s', rs') \xrightarrow{st_2} (te'', s'', rs'')$.

The rules for the operational semantics of statements are presented in Table 1, where $te[t/v]$ denotes the type environment that is obtained from te by assigning t to v, the state $s[c/v]$ and register state $rs[c/r]$ are defined accordingly.

Let us illustrate the semantics by using the "reg with reset: $\neq \bot$, true" rule: At first, $rs(r) \neq \bot$ implies that the value of the register r in the beginning of the current clock cycle is defined. Then the state s is updated by assigning $rs(r)$ to r, resulting into the new state s'. Moreover, because $s(e_1) = true$, the register state rs is updated by assigning $s(e_2)$ to r, resulting into the new register state rs'. In other words, the value of r is reset, that is, the value of r will be $s(e_2)$ in the beginning of the next clock cycle.

An *input trace* of the module A of length m is a function $it : \mathsf{InPorts(A)} \to \mathsf{BitVec}^{\mathsf{m}}$ such that $it(x)$ is a sequence of bit vectors and the length of $it(x)$ is m, for every $x \in \mathsf{InPorts(A)}$. Intuitively, for every $1 \leq i \leq m$, $it(x)[i]$ is the input value of the input port x at the i-th clock cycle.

Let ss be the sequence of statements in A and it be an input trace of A of length m. Then the semantics of A with respect to it is defined by the relations

$$(te, s, rs) \xrightarrow{A, it, j, m} (te', s', rs'), \text{ for } 0 \leq j \leq m.$$

$$\text{skip}\ \frac{-}{(te, s, rs) \xrightarrow{\text{skip}} (te, s, rs)} \qquad \text{wire}\ \frac{te' = te[t/v] \qquad s' = s[\bot/v]}{(te, s, rs) \xrightarrow{\text{wire } v:t} (te', s', rs)}$$

$$\text{reg: } \bot\ \frac{rs(r) = \bot \qquad te' = te[t/r] \qquad s' = s[\bot/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c} (te', s', rs)} \qquad \text{reg: } \neq \bot\ \frac{rs(r) \neq \bot \qquad s' = s[rs(r)/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c} (te, s', rs)}$$

$$\text{reg with reset: } \bot\text{, false}\ \frac{rs(r) = \bot \qquad s(e_1) = false \qquad te' = te[t/r] \qquad s' = s[\bot/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c \text{ with: reset => } (e_1,e_2)} (te', s', rs)}$$

$$\text{reg with reset: } \bot\text{, true}\ \frac{rs(r) = \bot \qquad s(e_1) = true \qquad te' = te[t/r] \qquad s' = s[\bot/r] \qquad rs' = s[s(e_2)/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c \text{ with: reset => } (e_1,e_2)} (te', s', rs')}$$

$$\text{reg with reset: } \neq \bot\text{, false}\ \frac{rs(r) \neq \bot \qquad s(e_1) = false \qquad s' = s[rs(r)/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c \text{ with: reset => } (e_1,e_2)} (te, s', rs)}$$

$$\text{reg with reset: } \neq \bot\text{, true}\ \frac{rs(r) \neq \bot \qquad s(e_1) = true \qquad s' = s[rs(r)/r] \qquad rs' = rs[s(e_2)/r]}{(te, s, rs) \xrightarrow{\text{reg } r:t,c \text{ with: reset => } (e_1,e_2)} (te, s', rs')}$$

$$\text{node}\ \frac{te' = te[te(e)/v] \qquad s' = s[s(e)/v]}{(te, s, rs) \xrightarrow{\text{node } v=e} (te', s', rs)} \qquad \text{connect: non-register}\ \frac{v \notin \text{Regs(A)} \qquad s' = s[s(e)/v]}{(te, s, rs) \xrightarrow{v<=e} (te, s', rs)}$$

$$\text{connect: register}\ \frac{v \in \text{Regs(A)} \qquad rs' = rs[s(e)/v]}{(te, s, rs) \xrightarrow{v<=e} (te, s, rs')} \qquad \text{invalid}\ \frac{s' = s[0^{wd(te(v))}/v]}{(te, s, rs) \xrightarrow{v \text{ is invalid}} (te, s', rs)}$$

Table 1. Operational semantics of LoFIRRTL statements.

Intuitively, the relation $(te, s, rs) \xrightarrow{A,it,j,m} (te', s', rs')$ denotes that (te, s, rs) is the configuration resulted from processing the first $m-j$ clock cycles of the input trace and (te', s', rs') is obtained after processing the remaining j clock cycles of the input trace . The relation $(te, s, rs) \xrightarrow{A,it,j,m} (te', s', rs')$ can inductively interpreted as follows:

- First, the input ports in the $(m - j + 1)$-th clock cycle get signals from the input trace it, namely, replace the value of each input port x in the state s with $it(x)[m - j + 1]$, resulting into a new state s_1.
- Then, the statement sequence ss of the module A is executed one time in one clock cycle from the configuration (te, s_1, rs), resulting into a new configuration (te'', s'', rs'').
- Finally, the configuration (te', s', rs') is obtained by executing the statement sequence ss in the last $j - 1$ clock cycles from the configuration (te'', s'', rs'').

Formally, the relation $(te, s, rs) \xrightarrow{A,it,j,m} (te', s', rs')$ is inductively defined as follows:

- If $j = 0$, then $(te, s, rs) \xrightarrow{A,it,j,m} (te, s, rs)$.
- Otherwise, if $(te, s_1, rs) \xrightarrow{ss} (te'', s'', rs'')$ and $(te'', s'', rs'') \xrightarrow{A,it,j-1,m} (te', s', rs')$, then $(te, s, rs) \xrightarrow{A,it,j,m} (te', s', rs')$, where s_1 is the state obtained from the state s by assigning the value $it(x)[m - j + 1]$ to the input port x for each $x \in \text{InPorts(A)}$.

3.1.2 General LoFIRRTL. W.l.o.g., we assume that the circuit A contains at least two instances of modules, namely, the top-level module A contains at least one `inst of` statement. (Note that the top-level module A contributes one instance to the circuit A without using any `inst of` statements.)

Moreover, we assume that the module dependency graph, which comprises the edges (B, C) if a statement `inst y of C` occurs in the module B for some y, is acyclic. This ensures that there are no mutual dependencies between modules in the circuit A, similar to mutual class inheritance in object-oriented programming languages.

For the sake of simplicity, instead of directly defining an operational semantics for the circuit A, we choose to flatten the hierarchies of the modules in the circuit A into a circuit containing a single module M_A, i.e., an `inst of`-free LoFIRRTL module, similar to procedure inlining in software programs. The resulting module M_A is in turn transformed into a pseudo-SSA form module M'_A whose operational semantics has been defined above. Finally, we define the operational semantics of the circuit A as that of the module M'_A.

In the sequel, we show how to flatten the hierarchies of the modules in circuit A into a module M_A. We define the set of module instances of the circuit A, denoted by $MI(A)$, inductively as follows. An element of $MI(A)$ is a pair (id, B), which denotes an instance of the module B with the identifier id, i.e., `inst id of B` is a statement in the circuit A.

- $(A, A) \in MI(A)$, that is, an instance of the top-level module A is in $MI(A)$. Note that we use A as the identifier of the instance of the module A, since there is a unique instance of the top-level module A.
- If $(x, B) \in MI(A)$, and the module B has a statement `inst y of C`, then $(x.y, C) \in MI(A)$, where the prefix "x." is added to avoid name conflict.

For each module instance $(id, B) \in MI(A)$, we inductively construct a module $M_{(id,B)}$ as follows:

- If B is an `inst of`-free module, then $M_{(id,B)}$ is obtained from the module B by replacing all identifiers z with $id.z$.
- Otherwise, suppose that the module $M_{(id.y,C)}$ has been constructed for each statement `inst y of C` in the module B. Then $M_{(id,B)}$ is obtained from the module B by first replacing all identifiers z with $id.z$, then replacing each statement `inst id.y of C` with the statement sequence in $M_{(id.y,C)}$.

Finally, the `inst of`-free LoFIRRTL module M_A is $M_{A,A}$.

Furthermore, M_A can be turned into M'_A in pseudo-SSA form by constructing the dependency graph G_{M_A}, building a topological sorting of G_{M_A}, and reordering the generalized connect statements of M_A according to the topological sorting.

3.2 Extraction of the OCaml LoFIRRTL Interpreter

The operational semantics for LoFIRRTL formalized in Coq not only facilitates a precise understanding of the meanings of LoFIRRTL programs, it can also be turned into an interpreter where the LoFIRRTL programs can be executed, by utilizing the extraction mechanism of Coq [**?**]. Our goal is to have an *end-to-end* interpreter for LoFIRRTL programs, that is, given a LoFIRRTL program as well as an input trace, the interpreter first parses the input program, then executes the program on the input trace and produces a sequence of outputs. In the sequel, we have a closer look at the extraction of the LoFIRRTL interpreter from the Coq programs.

Since the extraction of the OCaml code out of the operational semantics formalized in Coq is straightforward, it is relatively easy to obtain an engine in OCaml that is able to execute a LoFIRRTL program on an input trace. Note that the input programs of the execution engine are required to be in pseudo-SSA form, since the operational semantics in Section 3.1 is based on the assumption that the LoFIRRTL programs are in pseudo-SSA form. Nevertheless, this requirement may not be satisfied by the LoFIRRTL programs produced by the lowering transformations in general. Therefore, LoFIRRTL programs should be transformed into a pseudo-SSA form before being processed by the execution engine.

To tackle this transformation, we adapt a depth-first search (DFS) algorithm from the graph library in the mathematical components repository [**??**] into a topological sorting algorithm so that given a directed graph, the algorithm either reports that the input graph is acyclic and outputs a topological sorting of the acyclic graph, or reports that the input graph is cyclic and outputs a cycle. Moreover, accordingly, we prove the correctness of the topological sorting algorithm by adapting the correctness proof of the DFS algorithm. We should remark that the original DFS algorithm will always produce a sequence of vertices, which is a topological order iff the graph is acyclic. However, we want to check for program errors and therefore extend the standard DFS algorithm with a check for cycles (which is not tremendously difficult but not available in the standard library). Our algorithm therefore has the additional guarantee: if it produces a sequence of vertices as output, that must be a topological order of the graph; if it produces a cycle, the graph must be cyclic. Equipped with the topological sorting algorithm, we transform a LoFIRRTL circuit A into a pseudo-SSA form as follows: At first, we build the dependency graph G_A as defined in Section 3.1. Then, we compute a topological sorting of G_A. Finally, we reorder the generalized connect statements in A according to the topological sorting.

Another building block of the end-to-end interpreter is a parser for LoFIRRTL programs generated using `ocamlyacc`. The parser relies on the OCaml types for LoFIRRTL constructs that are extracted from the Coq code that formalizes the syntax of LoFIRRTL. We implement the parser by defining the grammar of LoFIRRTL programs in the BNF-like form in `ocamlyacc`, then specifying the production rules to attach the grammar constructors to the OCaml types in semantic-actions. The evaluation of the semantic-actions will produce an AST to represent the LoFIRRTL program in the OCaml types, which are then taken by the execution engine.

Finally, the operational semantics defined in Section 3.1 iterates over the sequence of statements in each module starting from the top-level module of the given circuit. The iteration can be implemented in Coq in two different ways: non-tail recursion or tail recursion, that is, either process the first statement before recursively processing the remaining statements, or vice versa, namely, before processing the last statement, recursively process all the preceding statements. It turns out that the OCaml code extracted from the non-tail recursion is much more efficient than that from the tail recursion. Therefore, in view of the efficiency of the extracted LoFIRRTL interpreter, we choose to implement the operational semantics in Coq in a non-tail recursive way.

3.3 Validation of the OCaml LoFIRRTL Interpreter

Benchmarks. To extensively evaluate the OCaml LoFIRRTL Interpreter, we mainly collect LoFIRRTL circuits from the Chisel/FIRRTL execution engine–Treadle[4] and the Chisel Book–Digital Design with Chisel [**?**], because Treadle is more likely to execute correctly on them so that the correctness testing of the formalized operational semantics has high confidence.

There are 55 LoFIRRTL circuits inside the "chipsalliance/treadle/samples" directory of Treadle, among which 18 LoFIRRTL circuits are excluded because they are not self-contained (i.e., dependent upon external modules), or contain undefined semantics for memories, or have no output ports or cannot be handled by Treadle due to cyclical connection of ports. Thus, we consider the remaining 37 LoFIRRTL circuits from Treadle, among which three circuits have some `inst of` statements, i.e., hierarchies of the modules.

There are 51 Chisel examples in Scala inside the "schoeberl/chisel-book/src/main/scala" directory of the Chisel Book. These 51 Chisel examples are transformed into 77 LoFIRRTL circuits by applying the convertor ChiselStage within the Chisel compiler framework to each subclass of the class `Module` in Scala. (Note that some Chisel examples contain multiple such subclasses, thus yield

[4] https://github.com/chipsalliance/treadle.

Table 2. Results of conformance testing

Benchmark suit	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
Treadle (37 circuits)	38.08	12.18	3.13	7.16	✓
Chisel Book (77 circuits)	68.86	139.63	31.31	5.81	✗(1)
[?] (4 circuits)	15.00	18.05	0.60	29.81	✓
Our own (2 circuits)	50.50	188.80	62.52	3.20	✓
Average	**57.21**	**97.86**	**22.45**	**6.95**	

multiple LoFIRRTL circuits.) Among 77 LoFIRRTL circuits, there are 67 `inst of`-free LoFIRRTL circuits and 10 none-`inst of`-free LoFIRRTL circuits.

Besides the above LoFIRRTL circuits, we also collect all the four LoFIRRTL circuits from [?] which have been tested using a formal Agda-based Interpreter designed for a subset of LoFIRRTL, and design two additional LoFIRRTL circuits in consideration of the branch coverage of the LoFIRRTL OCaml Interpreter and different hierarchical structures of the modules.

In summary, we use 120 (37+77+4+2) LoFIRRTL circuits all of which can be handled by Treadle.

Testing method. Since the input LoFIRRTL circuits to our LoFIRRTL OCaml Interpreter should be in the pseudo-SSA form while all the collected LoFIRRTL circuits are not necessary in a pseudo-SSA form, to resolve this issue, we preprocess them based on our verified topological sorting algorithm, as described above. Furthermore, LoFIRRTL circuits themselves require input signals to execute. For this purpose, we implement a deriver in OCaml which first automatically identifies all the input ports and their data types for each LoFIRRTL circuit and then randomly generates 100 input traces for the LoFIRRTL circuit. Finally, for each given LoFIRRTL circuit and each its input trace, we compare the equivalence of output ports produced by our LoFIRRTL OCaml Interpreter and Treadle. We do not compare with the Agda-based Interpreter because it cannot directly execute LoFIRRTL circuits.

Testing results. The summary of conformance testing is reported in Table 2. Column (#LOC) shows the average number of locations for each benchmark suit. Column (Treadle) shows the average execution time t_1 of Treadle in microsecond (μs) for each benchmark suit. Column (Ours) shows the average execution time t_2 of our LoFIRRTL OCaml Interpreter in μs for each benchmark suit. Column (Speedup) is calculated via $\frac{t_1 - t_2}{t_2}$ for each benchmark suit. Column (Result) shows the comparison result of output ports between Treadle and our LoFIRRTL OCaml Interpreter under the 100 input traces, where ✓ indicates that the values of the output ports produced by two tools are the same and ✗(n) indicates that the values of the output ports of n benchmarks produced by two tools are different. The last row (Average) shows the average results over all 120 benchmarks. Detailed results are given in the supplementary material due to space limits.

Overall, our LoFIRRTL OCaml Interpreter successfully executes all the 120 benchmarks and produces the same values of the output ports as Treadle except for one benchmark `BundleVec` taken from the Chisel Book. This inconsistency is caused by the undefined semantics of the `validif` expressions. More specifically, the LoFIRRTL specification states that `validif (e1,e2)` evaluates to the expression `e2` if the conditional expression `e1` evaluates to true, otherwise undefined. In Treadle, `validif (e1,e2)` evaluates to `e2` regardless of the conditional expression `e1`, while in our operational semantics, `validif (e1,e2)` evaluates to `e2` if `e1` evaluates true, otherwise 0.

In terms of efficiency, our LoFIRRTL OCaml Interpreter is significantly faster (6.95 times on average) than Treadle on almost all the benchmarks except for one benchmark `HardwareHashMaker`

from Treadle. After an in-depth analysis, we found that `HardwareHashMaker` has 20 64-bit multiplication operations significantly more than the other benchmarks, our LoFIRRTL OCaml Interpreter is extracted from the bit-wise multiplication in Coq while Treadle uses integer multiplication. Though we could adopt integer multiplication as well, we chose bit-wise multiplication based on the bit-vector library in Coq whose correctness has been formally proved.

In order to have some measure of the effectiveness of our testing, we analyzed the branch coverage of our LoFIRRTL OCaml Interpreter. All the 120 benchmarks together cover all the 51 branches, as well as modules with instances of `inst of`-free modules and none-`inst of`-free modules. (Note that only 36 out of 51 branches and modules with instances of `inst of`-free modules are covered without our own two benchmarks.)

4 FORMALIZATION OF HIFIRRTL TRANSFORMATIONS

In this section, we show how the lowering transformations of HiFIRRTL, which were described informally in Section 2.2, are formalized in Coq. Specifically, we consider the formalization of the six lowering transformations, namely, "InferTypes", "InferWidths", "InferResets", "ExpandConnects", "ExpandWhens", and "LowerTypes".

For simplicity, let us focus on a HiFIRRTL circuit A that contains no `inst of` statements, that is, circuit A contains exactly one top-level module A.

The formalization utilizes a structure called *circuit diagrams* defined below.

Definition 4.1 (Circuit diagrams). A *circuit diagram* D is a triple (ss, ce, cm), where ss is a sequence of statements, ce is a component environment that maps each component to its type, and cm is a connect mapping that maps each component to an expression that is connected to it.

Note that the types in the range of the component environment ce in Definition 4.1 are not necessarily ground types, that is, they are aggregate types in general, since HiFIRRTL may contain high-level constructs, e.g. vectors and bundles. Intuitively, ss is the sequence of statements in a HiFIRRTL program, while ce and cm extract some necessary information from the sequence ss of statements to facilitate the formalization of the lowering transformations. Specifically, the component environment ce extracts the type information from the sequence ss of statements and the connect mapping cm extracts for each component the expression that is connected to the component according to the last-connect semantics. As we will see later on, the sequence ss of statements will be updated by the "ExpandConnects", "ExpandWhens", and "LowerTypes" passes, and the component environment ce will be modified by the "InferTypes", "InferWidths", "InferResets", and "LowerTypes" passes, while the component environment cm will only be changed by the "ExpandWhens" pass, since it is the only pass that deals with the last-connect semantics.

Before applying the lowering transformations, the module A will be first transformed into a circuit diagram $D_A = (ss_0, ce_0, cm_0)$, where ss_0 is the sequence of statements in the module A, ce_0 is the component environment obtained from the component declarations in the module A, possibly with the types of some components missing or partial, and cm_0 is the trivial mapping that maps each component to $\perp$. (Intuitively, this means that none of the components have been connected to so far.) The missing or partial types in the component environment ce_0 will be completed by the lowering transformations "InferTypes" and "InferWidths".

In the sequel, we utilize the concept of circuit diagrams to illustrate how the six lowering transformations are formalized in Coq.

4.1 Formalization of InferTypes

We formalize the "InferTypes" pass by defining a Coq function `infer_type`, which iterates over the statements in the sequence ss and updates the component environmen ce. For instance, the

following rule specifies that when the function `infer_type` is applied to a wire statement `wire` $x : t$, it updates the component environmen ce by assigning $(t, Wire)$ to the wire x (where $Wire$ denotes the kind of the component x), resulting into the component environment ce',

$$\frac{ce' = ce \uplus \{x : (t, Wire)\}}{ce \vdash \texttt{infer_type}(\texttt{wire}\ x : t) \rightsquigarrow ce'}.$$

Moreover, the function `infer_type` behaves similarly on `reg`, `input` and `output` port statements. Finally, on a node statement `node` $x = e$, `infer_type` first calculates the type of the expression e, denoted by t_e, then assigns $(t_e, Node)$ to the node x, resulting into the component environment ce',

$$\frac{ce \vdash t_e = \texttt{type_of_expr}(e) \qquad ce' = ce \uplus \{v : (t_e, Node)\}}{ce \vdash \texttt{infer_type}(\texttt{node}\ v = e) \rightsquigarrow ce'}.$$

Note that the function `infer_type` will not update the component environment ce when it is applied to connect statements. Nevertheless, in this case, as specified in the following two rules, it checks whether the two types of both sides of the connect statement are equivalent. By type equivalence, we mean that the structures of the types are the same, but their bit widths may be missing or different. For instance, the types `UInt`, `UInt<1>`, and `UInt<2>` are seen as equivalent. If the two types are equivalent, then the component environment ce is kept, otherwise, an error (indicated by $\perp$) will be issued.

$$\frac{ce(v) = (t_v, -) \qquad ce \vdash t_e = \texttt{type_of_expr}(e) \qquad \texttt{equiv}(t_v, t_e)}{ce \vdash \texttt{infer_type}(v <= e) \rightsquigarrow ce}$$

$$\frac{ce(v) = (t_v, -) \qquad ce \vdash t_e = \texttt{type_of_expr}(e) \qquad \neg\texttt{equiv}(t_v, t_e)}{ce \vdash \texttt{infer_type}(v <= e) \rightsquigarrow \perp}$$

After the function `infer_type` has been applied to all the statements in the sequence ss, if the bit width of a component is still missing, then the zero width will be added for the component in the component environment ce.

4.2 Formalization of InferWidths

We formalize the "InferWidths" pass by defining a Coq function `infer_width`, which iterates over the statements in the sequence ss and updates the component environment ce, so that the bit widths of all the components with undefined bit widths can be determined. The rationale behind the `infer_width` function is stated as follows: After applying the `infer_type` function to all the statements in the sequence ss, for each component x,

- if the bit width of the component x is nonzero, that is, a bit width is available in its declaration statement, then the function `infer_width` will *not* change the bit width of the component x,
- otherwise, the function `infer_width` chooses the maximum width of the expressions that are connected to the component x.

Note that the last-connected semantics is ignored here. The behavior of the function `infer_width` on a connect statement is defined by the following rule,

$$\frac{ce(x) = (t_x, \kappa) \qquad ce \vdash t_e = \texttt{type_of_expr}(e) \qquad ce' = ce \uplus \{x : (\texttt{max_width}(t_x, t_e), \kappa)\}}{ce \vdash \texttt{infer_width}(x <= e) \rightsquigarrow ce'}.$$

Moreover, the function `infer_width` uses another Coq function `max_width`, which, given two aggregate types, compares for each field their widths in the aggregate types, and chooses the larger width. Finally, the function `infer_width` processes the partial connect statements similarly.

4.3 Formalization of InferResets

We formalize the "InferResets" pass by defining a Coq function `infer_reset`, which iterates over the statements in the sequence ss and updates the component environment ce, so that the concrete reset types of all abstract reset signals can be inferred. The rationale behind the function `infer_reset` is stated as follows: After applying the function `infer_type` to all the statements in the sequence ss, for each component x of type `Reset`,

- if only components of type `AsyncReset` are connected to the component x, then the function `infer_reset` updates the type of the component x to `AsyncReset`,
- if x `is invalid` is a statement in the sequence ss or only components of type `UInt<1>` are connected to the component x, then the function `infer_reset` updates the type of the component x to the type `UInt<1>`,
- if two components of type `AsyncReset` and `UInt<1>` respectively are both connected to the component x, then the function `infer_reset` reports an error.

The behavior of the `infer_reset` function on a connect statement is defined by the following two rules,

$$\frac{ce(x) = (t_x, \kappa) \qquad ce \vdash t_e = \texttt{type_of_expr}(e) \qquad \neg\texttt{conflict}(t_x, t_e) \qquad ce' = ce \uplus \{x : (t_e, \kappa)\}}{ce \vdash \texttt{infer_reset}(x <= e, ce) \rightsquigarrow ce'}$$

and

$$\frac{ce(x) = (t_x, \kappa) \qquad ce \vdash t_e = \texttt{type_of_expr}(e) \qquad \texttt{conflict}(t_x, t_e)}{ce \vdash \texttt{infer_reset}(x <= e, ce) \rightsquigarrow \bot}.$$

Note that the function `infer_reset` uses the predicate `conflict` to decide whether two reset types are conflicting, that is, one is `AsyncReset` and the other is `UInt<1>`. Moreover, here we take `is invalid` statements as connect statements connected to a synchronous reset component (i.e. of type `UInt<1>`).

4.4 Formalization of ExpandConnects

The "ExpandConnects" pass expands the connections between aggregate types into those between ground types. We define two Coq functions `expand_expr` and `expand_cnct` that expand the expressions and the connect statements, respectively. The rationale behind the functions `expand_expr` and `expand_cnct` is to break aggregate types in expressions and connect statements recursively into their fields. Note that the function `expand_cnct` expands both connect and partial connect statements. The functions `expand_expr` and `expand_cnct` modify the sequence ss of statements, instead of component environment ce.

The function `expand_expr` expands an expression e into a list of expressions of ground types recursively by utilizing `type_of_expr`(e). Note that only expressions using the operators of multiplex `mux` and `validif`, or references can have aggregate types, while the expressions formed by using binary operators e.g. *add* do not. The function `expand_expr` expands the `mux` and `validif` expressions by the following two rules,

$$\frac{els = \texttt{ls_mux}(e_c, els_t, els_f) \qquad els_t = \texttt{expand_expr}(e_t) \qquad els_f = \texttt{expand_expr}(e_f)}{ce \vdash \texttt{expand_expr}(\texttt{mux}(e_c, e_t, e_f)) \rightsquigarrow els}$$

and

$$\frac{els = \texttt{ls_validif}(e_c, els_t) \qquad els_t = \texttt{expand_expr}(e)}{ce \vdash \texttt{expand_expr}(\texttt{validif}(e_c, e)) \rightsquigarrow els}.$$

The first rule uses a Coq function `ls_mux` that combines two sequences of expressions els_t and els_f with the same condition e_c, resulting into a list of ground-type expressions

$$\texttt{mux}(e_c, els_{t,1}, els_{f,1}), \cdots, \texttt{mux}(e_c, els_{t,m}, els_{f,m}),$$

where $els_{t,i}$ (resp. $els_{f,i}$) denotes the i-th expression in the sequence of expressions els_t (resp. els_f). The function `ls_validif` in the second rule is defined similarly. Finally, `expand_expr`(r) expands a reference r straightforwardly and creates a list of expressions of ground types according to `type_of_expr`(r).

Equipped with the `expand_expr` function, the function `expand_cnct` works as follows: Given a connect statement (x <= e), it applies the function `expand_expr` to the component x and the expression e respectively, producing two sequences of expressions of ground types, then constructs a sequence of connect statements by combining the expressions in the two sequences. The behavior of the `expand_cnct` function on a connect statement is defined by the following rule,

$$\frac{ss' = \texttt{ls_cnct}(els_l, els_r) \qquad els_l = \texttt{expand_expr}(x) \qquad els_r = \texttt{expand_expr}(e)}{ce \vdash \texttt{expand_cnct}(x <= e) \rightsquigarrow ss'}.$$

The function `expand_cnct` uses a Coq function `ls_cnct` that combines the two sequences els_l and els_r of expressions into a sequence ss' of connect statements between ground types, that is,

$$\text{the statements } els_{l,1} <= els_{r,1}, \cdots, els_{l,m} <= els_{r,m},$$

where m is the length of the two sequences, and $els_{l,i}$ (resp. $els_{r,i}$) denotes the i-th expression of the sequence els_l (resp. els_r).

The function `expand_cnct` handles the `is invalid` statements in the same way as connect statements. It produces from a `is invalid` statement a sequence of `is invalid` statements for its fields.

4.5 Formalization of ExpandWhens

The "ExpandWhens" pass ensures that every component in a module is connected to at most once. It reformulates the conditions in the `when` statements by multiplex expressions and removes the `when` statements. Moreover, for each component, it removes the non-last (redundant) connect statements for the component. We define two functions `expand_branch` and `combine_branches` to formalize the "ExpandWhens" pass. The function `expand_branch` scans through the statement sequence ss, copies the declaration statements, and updates the component environment cm when processing connect, `is invalid`, and `when` statements. The function `combine_branches` is used to combine the connect mappings for branches of `when` statements. The `expand_branch` function is defined by the following four rules, where *declare* x represents a declaration statement for x, e.g. `wire x: t`.

$$\frac{\texttt{expand_branch}(ss, cm_0) = (ss_{\text{dcl}}, cm)}{\texttt{expand_branch}(ss; \textit{declare } x, cm_0) = (ss_{\text{dcl}}; \textit{declare } x, cm[\bot/x])}$$

$$\frac{\texttt{expand_branch}(ss, cm_0) = (ss_{\text{dcl}}, cm)}{\texttt{expand_branch}(ss; x <= e, cm_0) = (ss_{\text{dcl}}, cm[e/x])}$$

$$\frac{\texttt{expand_branch}(ss, cm_0) = (ss_{\text{dcl}}, cm)}{\texttt{expand_branch}(ss; v \texttt{ is invalid}, cm_0) = (ss_{\text{dcl}}, cm[\texttt{invalid}/v])}$$

$$\frac{\texttt{expand_branch}(ss, cm_0) = (ss_{\text{dcl}}, cm) \qquad \begin{array}{c}\texttt{expand_branch}(ss^{\mathsf{T}}, cm) = (ss^{\mathsf{T}}_{\text{dcl}}, cm^{\mathsf{T}}) \\ \texttt{expand_branch}(ss^{\mathsf{F}}, cm) = (ss^{\mathsf{F}}_{\text{dcl}}, cm^{\mathsf{F}})\end{array}}{\texttt{expand_branch}(ss; \texttt{when c then } ss^{\mathsf{T}} \texttt{ else } ss^{\mathsf{F}}, cm_0) = (ss_{\text{dcl}}; ss^{\mathsf{T}}_{\text{dcl}}; ss^{\mathsf{F}}_{\text{dcl}}, \texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}}))}$$

Note that for a statement `when c then` ss^{T} `else` ss^{F}, the function `expand_branch` first processes the statements preceding this statement and obtains a connect mapping cm, then uses cm to process ss^{T} and ss^{F} respectively, resulting into the connect mappings cm^{T} and cm^{F} respectively. Finally, cm^{T} and cm^{F} are combined into one mapping by `combine_branches`.

The function `combine_branches` combines two connect mappings into one by the following fix rules where an expression is constructed for each component, denoting the fact that the expression is connected to it according to the last-connect semantics. If both connect mappings assign the same value to a component, then no multiplexer or `validif` is needed in the combined expression.

$$\frac{cm^{\mathsf{T}}(x) = cm^{\mathsf{F}}(x) = e}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = e}$$

$$\frac{cm^{\mathsf{T}}(x) = cm^{\mathsf{F}}(x) = \texttt{invalid}}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = \texttt{invalid}}$$

If the two connect mappings assign different values to a component, then a multiplexer or `validif` expression is obtained.

$$\frac{cm^{\mathsf{T}}(x) = e^{\mathsf{T}} \qquad cm^{\mathsf{F}}(x) = e^{\mathsf{F}} \qquad e^{\mathsf{T}} \neq e^{\mathsf{F}}}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = \texttt{mux}(\mathsf{c}, e^{\mathsf{T}}, e^{\mathsf{F}})}$$

$$\frac{cm^{\mathsf{T}}(x) = e^{\mathsf{T}} \qquad cm^{\mathsf{F}}(x) = \texttt{invalid}}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = \texttt{validif}(c, e^{\mathsf{T}})}$$

If only one branch declared the component, ignore the condition; if at least one branch did declare the component but erroneously did not connect it, register that it is still not (always) connected:

$$\frac{cm^{\mathsf{T}}(x) = e^{\mathsf{T}} \qquad cm^{\mathsf{F}}(x) \text{ is undefined}}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = e^{\mathsf{T}}}$$

$$\frac{cm^{\mathsf{F}}(x) = \bot}{\texttt{combine_branches}(cm^{\mathsf{T}}, cm^{\mathsf{F}})(x) = \bot}$$

For the last three rules, a symmetric variant exists where the role of the true and false branches are swapped.

Moreover, to initialize, the empty statement sequence does not change a connect map, specified by the following rule:

$$\frac{}{\texttt{expand_branch}(\emptyset, cm_0) = (\emptyset, cm_0)}$$

Finally, the statement sequence is updated by concatenating the statement sequence generated by the function `expand_branch` and the sequence of connect and `is invalid` statements corresponding to the connect mapping generated by the function `expand_branch`. If in the end some component is still assigned the value $\bot$ by the connect mapping, an appropriate error message can be generated.

4.6 Formalization of LowerTypes

The "LowerTypes" pass flattens each component of an aggregate type into a collection of components of ground types. We formalize this pass by a function `lower_types`. The rationale behind the function `lower_types` is that except for the memories and module instances, each component of an aggregate type is replaced by a collection of new components that have ground types, one for each field, where *paired identifiers* are introduced for naming the new components.

A paired identifier is of the form (id_1, id_2), where id_1 is the identifier of a component of aggregate type, and id_2 is the name of a field in the aggregate type. The function `lower_types` processes the

statements in the sequence ss and updates the sequence ss and the component environment ce. The behavior of the function `lower_types` on the declaration statements is defined by the rule below, where $declare\ x$ denotes the declaration statement for a component x, and $ce = (t, \kappa)$ (t is the aggregate type and κ is the kind, e.g. `wire`), moreover, a function `flatten` is used to flatten $declare\ x$ into a sequence of declaration statements. Specifically, let the fields in t be $f_1, \cdots, f_n$, of ground types $t_1, \cdots, t_n$ respectively, then the `flatten` function generates from $(x, (t, \kappa))$ a sequence of declaration statements $\kappa\ (x, f_1) : t_1, \cdots, \kappa\ (x, f_n) : t_n$.

$$\frac{ce(x) = (t, \kappa) \qquad ss' = \texttt{flatten}(x, (t, \kappa)) \qquad ce' = ce \uplus \{(x, f_1) : (t_1, \kappa), \ldots, (x, f_n) : (t_n, \kappa)\}}{ce \vdash \texttt{lower_types}(declare\ x) \rightsquigarrow (ss', ce')}.$$

Note that the "LowerTypes" pass should be applied after the "ExpandConnects" pass. Therefore, before applying the function `lower_types` to the statements in the sequence ss, both sides of the connect statements are already in ground types. Then when the `lower_types` function processes a connect statement, the references to the component fields in this statement should be replaced by the corresponding paired identifiers. Thus in this case, all the three components of the circuit diagram, namely, the statement sequence, the component environment, the connect mapping, are all updated to reflect the replacements of component fields by paired identifiers.

5 SPECIFICATION AND VERIFICATION OF FUNCTIONAL CORRECTNESS OF ExpandConnects AND ExpandWhens

The goal of this section is to prove that the Coq implementations of two most involved lowering transformations "ExpandConnects" and "ExpandWhens" are functionally correct. Recall that these two transformations were implemented as two functions `expand_cnct` and `expand_branch` in Section 4. In the sequel, we first specify the correctness properties that are supposed to be satisfied by the two functions. Then, we prove that they indeed satisfy these correctness properties. Note that different from the verification of software compilers, e.g. CompCert, we are not proving the preservation of semantics for the compilation passes. This difference is attributed to the fact that it is hard, if not impossible, to define an executable semantics for HiFIRRTL, since some necessary information for the execution, e.g. bit widths of components, are still missing in HiFIRRTL programs.

5.1 Correctness of ExpandConnects

Let ss be the statement sequence before applying the function `expand_cnct` and ss_{cnct} be the sequence of connect statements in ss.

Specification. Then the correctness of `expand_cnct` is specified by the formula

$$\texttt{type_equivs}(\texttt{expand_cnct}(ss_{\text{cnct}})) \wedge \texttt{all_ground}(\texttt{expand_cnct}(ss_{\text{cnct}})).$$

The formula $\texttt{type_equivs}(\texttt{expand_cnct}(ss_{\text{cnct}}))$ specifies that for each connect statement generated by $\texttt{expand_cnct}(ss_{\text{cnct}})$, the types of its left-hand-side and right-hand-side expressions are equivalent. On the other hand, the formula $\texttt{all_ground}(\texttt{expand_cnct}(ss_{\text{cnct}}))$ specifies that all the expressions occurring in the connect statements generated by $\texttt{expand_cnct}(ss_{\text{cnct}})$ have ground types.

More specifically, the predicate $\texttt{type_equivs}(ss')$ is recursively defined and requires that each connect statement $x <= e$ in ss' satisfies that $\texttt{type_equiv}(\texttt{type_of_expr}(x), \texttt{type_of_expr}(e))$ hods, where $\texttt{type_equiv}(t_1, t_2)$ is the predicate that specifies the type equivalence of t_1 and t_2. On the other hand, the predicate $\texttt{all_ground}(ss')$ is recursively defined and requires that each connect statement $x <= e$ in ss' satisfies that $\texttt{ground}(\texttt{type_of_expr}(x)) \wedge \texttt{ground}(\texttt{type_of_expr}(e))$ holds, where the predicate $\texttt{ground}(t)$ checks that t is a ground type.

Verification. Recall that `expand_cnct` expands a connect statement between components of aggregate types by using the function `expand_expr` to expand each of the left-hand-side and right-hand-side expressions of the statement into a sequence of expressions of ground types. Since `expand_expr`(e) expands the expression e by following the structure of `type_of_expr`(e), we define a recursive function `expand_type`(t) that expands an aggregate type t into a list of ground types. Then we prove the following two lemmas to prove $ce \vdash$ `type_equivs`(`expand_cnct`(ss_{cnct})).

Lemma 5.1. *For every connect statement x <= e in ss_{cnct},*

$$ce \vdash \texttt{type_equiv}(\texttt{type_of_expr}(x), \texttt{type_of_expr}(e)) \rightarrow \texttt{all2}(\texttt{type_equiv}, \texttt{expand_type}(\texttt{type_of_expr}(x)), \texttt{expand_type}(\texttt{type_of_expr}(e))).$$

In Lemma 5.1, the `all2` function is used to check whether the predicate `type_equiv` is satisfied by every pairwise types in `expand_type`(`type_of_expr`(x)) and `expand_type`(`type_of_expr`(e)).

Lemma 5.2. *For every expression e,*

$$ce \vdash \texttt{map}(\texttt{type_of_expr}, \texttt{expand_expr}(e)) = \texttt{expand_type}\,(\texttt{type_of_expr}(e)).$$

Note that in Lemma 5.2, the `map` function is used to apply the function `type_of_expr` to every expression in `expand_expr`(e) to produce a list of types.

Finally, $ce \vdash$ `all_ground`(`expand_cnct`(ss_{cnct})) can be proved by the following lemma.

Lemma 5.3. *For every expression e,*

$$ce \vdash \texttt{all}(\texttt{ground}, \texttt{expand_type}(\texttt{type_of_expr}(e))).$$

Note that in Lemma 5.3, the `all` function is used to check whether the predicate `ground` is satisfied by every type in `expand_type`(`type_of_expr`(e))).

5.2 Correctness of ExpandWhens

Recall that the "ExpandWhens" pass was implemented by the function `expand_branch` in Section 4. The function `expand_branch` requires a mutual induction over statements and statement sequences, because each `when` statement may contain two statement sequences, one for each branch. Therefore, a correctness proof for `expand_branch` would also require such a mutual induction.

In the sequel, we first introduce a concept of expression trees to specify the correctness property of `expand_branch`, then prove that `expand_branch` satisfies the property.

Specification. We first define the concept of expression trees.

Definition 5.4 (Expression trees). An *expression tree* is either a binary tree whose internal nodes are annotated with conditions (usually, conditions found in `when` statements) and whose leaves are annotated with expressions, or a single-node tree that is annotated by one of three special constants, namely, `invalid`, `not connected`, and `undeclared`.

Intuitively, an expression tree is attached to a component x, it summarizes all the connections for x under different conditions, that is, it manifests the structure of the multiplexer expression that is obtained by weaving together all the expressions connected to x, according to the (possibly) nested structure of `when` statements. The special constants are used to denote the abnormal situations that x is either stated as `invalid`, or not connected in some cases, or undeclared.

The concept of expression trees serves as a basis for the specification of the functional correctness property of `expand_branch`. We define a function `expand_branch_one_var_sem` in Coq, which constructs an expression tree for each ground-type (sub-)component. It takes as parameters a statement sequence ss and a ground-type (sub-)component ref. It constructs the expression tree for

ref recursively, based on the nested structure of `when` statements and connect statements in *ss*. Let $\texttt{expand_branch}(ss, cm_0) = (ss', cm)$, then the correctness property of `expand_branch` is stated as

$$\bigwedge_{ref} cm(ref) = \texttt{expand_branch_one_var_sem}(ss, ref), \tag{1}$$

where ref ranges over all the ground-type (sub-)components of *ss*.

Verification. For the correctness proof, we define a predicate `expand_branch_sem_conform`, which, given the statement sequence of a module or of a `when` branch, check whether the implementation $\texttt{expand_branch}(ss, cm_0)$ satisfies the correctness property stated in Equation (1).

The proof of the validity of `expand_branch_sem_conform` requires a mutual induction over statement sequences and statements: If a statement sequence *ss* and a statement *s* satisfy the correctness property, then also the statement sequence $ss; s$. Moreover, a statement `when` $c : ss^{\mathsf{T}}$ `else` ss^{F} satisfies the correctness property if both ss^{T} and ss^{F} satisfy it. The mutual induction also requires us to define the two Coq datatypes mutually inductive; we cannot just use the sequence or list types from the standard Coq library. To be frank, the proof is more long than complicated.

6 RELATED WORK

In this section, we discuss some related work about compiler verification for software and hardware design languages.

The earliest work in compiler verification dates back to 1967, when [?] proved the correctness of a compiler that translates arithmetic expressions into stack machine code. Since then, many other proofs have been conducted, ranging from single compilation pass to sophisticated code optimizations of compilers that translate programs in high-level software or hardware programming languages into low-level and/or optimized code [?].

Verified compilations for software programming languages include CompCert for C [??????], Vélus for Lustre [?], Vellvm [??], Crellvm [?] for the LLVM Intermediate Representation, CakeML for ML [????], to cite a few. The interactive theorem prover Coq has been widely adopted to formalize semantics of programs, compilation passes and optimizations, based on which compilation correctness can be proved and executable interpreters and compilers can be automatically generated (cf. [?] for a curated list). For instance, [?] formalized and proved a compiler that translates programs in a simple functional language into code in an idealized assembly language. [??] formalized and proved compilers that translate programs in simply-typed or untyped lambda calculus to code in an idealized assembly language. CompCert is a verified compiler that translates programs in Clight (a large subset of the C programming language) to code in PowerPC assembly language.

The motivation to make hardware design more like software has promoted dozens of researches. The semantic challenges and formal semantics of hardware design languages, Verilog and VHDL, were discussed and studied by [??]. [?] introduced a new library to model and verify hardware circuits in Coq. [?] implemented a verified compiler that translates a high-level hardware description language, called Fe-Si, to RTL verilog, following the ideas behind CompCert. A toolbox was implemented in Coq dedicated to the specification and verification of synchronous sequential devices, based on which researchers finished circuits certification in type theory [?]. [?] implemented another Coq library, called Kami, enabling expressive and modular reasoning for hardware designs. To apply mechanization to high-assurance hardware, calculus of secure hardware description with its formal semantics and security type system is constructed in [?]. [?] proposed to describe hardware programs in Agda so that they can be simulated, tested, verified and synthesized. [?] carried out formal verification based on the analysis of sets of traces of transition systems for hardware description language. [?] proposed a proof-producing translator for the subset of Verilog.

[?] designed Veriformal, an executable formal model for most constructs of Verilog used in hardware designs, with which one can analyse and execute Verilog designs. Recently, an extension of verified ML compiler CakeML, called Lutsig, was developed for compiling a subset of RTL Verilog programs to netlist level programs for field-programmable gate array (FPGA) [??].

In this work, we focus on the formalization of FIRRTL, an intermediate representation for the Chisel hardware description language. It is argued by the Chisel team that FIRRTL is the key for bringing software levels of productivity to hardware development [?]. As mentioned in the introduction, the closest work to ours is [?], which formalized a coinductive denotational semantics in Agda for an idealized subset of LoFIRRT, based on a formalism called device calculus. Our work makes substantial progress from [?] in the following aspects: At first, the operational semantics of LoFIRRTL is defined for the first time. Moreover, extensive experiments are done to validate the semantics against Treadle, the reference interpreter for LoFIRRTL, by utilizing a benchmark suite comprising hundreds of programs that are collected from various sources. Furthermore, we go one step further by formalizing and verifying HiFIRRTL and its lowering transformations, thus laying down a solid foundation towards a fully verified FIRRTL compiler.

Translation validation proves the semantic equivalence between the high-level input program and the low-level output design [??], and has been successfully applied to several hardware program optimizations [?????]. While translation validation is less expensive than compiler verification, it offers weaker correctness guarantees, since the translation validation verifies the semantic equivalence for only one pair of an input program and its corresponding output. In contrast, verified compilers guarantee the semantic equivalence for every pair of high-level input programs and their corresponding low-level hardware designs.

7 CONCLUSION AND FUTURE WORK

In this work, we established a general framework called Vefirrtl where programs in FIRRTL, an intermediate representation language for hardware designs, and its lowering transformations, can be formally reasoned about. We defined for the first time an elegant operational semantics for Low FIRRTL, a restricted form of FIRRTL programs resembling RTL Verilog. Moreover, we extract an OCaml interpreter for LoFIRRTL and validate its consistency against Treadle, the mainstream FIRRTL reference interpreter, on 120 programs collected from various sources. Going one step further, we also formalize HiFIRRTL and its six key lowering transformations, specify and verify the functional correctness of two most involved transformations among them, namely, “ExpandConnects” and “ExpandWhens”. Vefirrtl paves the way towards a fully verified FIRRTL compiler, which is left as future work.

Supplementary Material of Submission #792

A MISSING DETAILS IN SECTION 3.1

The definitions of $te(e)$ and $s(e)$. For θ<n>(n') such that $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, $n > 0$, and n' is an integer, we define $bv_{\theta<n>(n')}$ as follows.

- For $n > 0$ and $n' \geq 0$ such that $n' < 2^n$, let $bv_{\texttt{UInt}<n>(n')}$ denote the bit vector of length n representing the unsigned integer n'.
- For $n > 0$ and an integer n' such that $-2^{n-1} \leq n' < 2^{n-1}$, let $bv_{\texttt{SInt}<n>(n')}$ denote the bit vector of length n representing the signed integer n'.

The function te and s from Cpnts(A) to GTypes are extended to expressions as follows.

- $te(\theta<n>(c)) = \theta<n>$ and $s(\theta<n>(c)) = bv_{\theta<n>(c)}$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$,
- let $te(e_1) = \theta<n_1>$ and $te(e_2) = \theta<n_2>$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, then $te(mux(e_0, e_1, e_2)) = \theta<\max(n_1, n_2)>$, moreover, if $s(e_0) = 1$, then $s(mux(e_0, e_1, e_2)) = bv_{\theta<n_0>(s(e_1))}$, otherwise, $s(mux(e_0, e_1, e_2)) = bv_{\theta<n_0>(s(e_2))}$,
- $te(validif(e_1, e_2)) = te(e_2)$, moreover, if $s(e_1) = 1$, then $s(validif(e_1, e_2)) = s(e_2)$, otherwise, $s(validif(e_1, e_2)) = \bot$,
- let $te(e_1) = \theta<n_1>$ and $te(e_2) = \theta<n_2>$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, then
  - $te(add(e_1, e_2)) = \theta<\max(n_1, n_2) + 1>$ and $te(sub(e_1, e_2)) = \theta<\max(n_1, n_2) + 1>$, moreover, $s(add(e_1, e_2)) = bv_{\theta<\max(n_1,n_2)+1>(s(e_1)+s(e_2))}$ and $s(sub(e_1, e_2)) = bv_{\theta<\max(n_1,n_2)+1>(s(e_1)-s(e_2))}$,
  - $te(mul(e_1, e_2)) = \theta<n_1 + n_2>$ and $s(mul(e_1, e_2)) = bv_{\theta<n_1+n_2>(s(e_1)\times s(e_2))}$,
  - if $\theta = \texttt{UInt}$, then $te(div(e_1, e_2)) = \texttt{UInt}<n_1>$ and $s(div(e_1, e_2)) = bv_{\texttt{UInt}<n_1>(s(e_1)\div s(e_2))}$, otherwise, $te(div(e_1, e_2)) = \texttt{SInt}<n_1 + 1>$ and $s(div(e_1, e_2)) = bv_{\texttt{SInt}<n_1+1>(s(e_1)\div s(e_2))}$,
  - $te(mod(e_1, e_2)) = \theta<\min(n_1, n_2)>$ and $s(mod(e_1, e_2)) = bv_{\theta<\min(n_1,n_2)>(s(e_1) \bmod s(e_2))}$,
  - $te(cp(e_1, e_2)) = \texttt{UInt}<1>$, moreover, $s(cp(e_1, e_2)) = 1$ iff $s(e_1)\ cp\ s(e_2)$, where $cp \in \{lt, leq, gt, geq, eq, neq\}$,
  - $te(dshl(e_1, e_2)) = \theta<n_1 + 2^{n_2} - 1>$ and $s(dshl(e_1, e_2)) = bv_{\theta<n_1+2^{n_2}-1>(s(e_1)\times 2^{s(e_2)})}$,
  - $te(dshr(e_1, e_2)) = \theta<n_1>$ and $s(dshr(e_1, e_2)) = bv_{\theta<n_1>(s(e_1)\div 2^{s(e_2)})}$,
  - $te(and(e_1, e_2)) = te(or(e_1, e_2)) = te(xor(e_1, e_2)) = \texttt{UInt}<\max(n_1, n_2)>$, moreover, $s(and(e_1, e_2))$ is the bitwise and of $bv_{\theta<\max(n_1,n_2)>(s(e_1))}$ and $bv_{\theta<\max(n_1,n_2)>(s(e_2))}$, $s(or(e_1, e_2))$ is the bitwise or of $bv_{\theta<\max(n_1,n_2)>(s(e_1))}$ and $bv_{\theta<\max(n_1,n_2)>(s(e_2))}$, $s(xor(e_1, e_2))$ is the bitwise xor of $bv_{\theta<\max(n_1,n_2)>(s(e_1))}$ and $bv_{\theta<\max(n_1,n_2)>(s(e_2))}$,
  - $te(cat(e_1, e_2)) = \texttt{UInt}<n_1 + n_2>$, moreover,
    - if either $\theta = \texttt{UInt}$, or $\theta = \texttt{SInt}$, $s(e_1) \geq 0$, and $s(e_2) \geq 0$, then $s(cat(e_1, e_2))$ is $bv_{\texttt{UInt}<n_1+n_2>(s(e_1)\times 2^{n_2}+s(e_2))}$,
    - if $\theta = \texttt{SInt}$, $s(e_1) \geq 0$, and $s(e_2) < 0$, then $s(cat(e_1, e_2))$ is $bv_{\texttt{UInt}<n_1+n_2>(s(e_1)\times 2^{n_2}+2^{n_2}+s(e_2))}$,
    - if $\theta = \texttt{SInt}$ and $s(e_1) < 0, s(e_2) \geq 0$, then $s(cat(e_1, e_2))$ is $bv_{\texttt{UInt}<n_1+n_2>((2^{n_1}+s(e_1))\times 2^{n_2}+s(e_2))}$,
    - if $\theta = \texttt{SInt}$ and $s(e_1) < 0, s(e_2) < 0$ then $s(cat(e_1, e_2))$ is $bv_{\texttt{UInt}<n_1+n_2>((2^{n_1}-s(e_1))\times 2^{n_2}+2^{n_2}-s(e_2))}$,
- let $te(e_1) = \theta<n_1>$ and $te(e_2) = \texttt{UInt}<n_2>$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, then
  - $te(dshl(e_1, e_2)) = \theta<n_1 + 2^{n_2} - 1>$ and $s(dshl(e_1, e_2)) = bv_{\theta<n_1+2^{n_2}-1>(s(e_1)\times 2^{s(e_2)})}$,
  - $te(dshr(e_1, e_2)) = \theta<n_1>$ and $s(dshr(e_1, e_2)) = bv_{\theta<n_1>(s(e_1)\div 2^{s(e_2)})}$,
- let $te(e_1) = \theta<n_1>$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, then
  - $te(andr(e_1)) = te(orr(e_1)) = te(xorr(e_1)) = \texttt{UInt}<1>$, moreover, $s(andr(e_1))$ (resp. $s(orr(e_1))$, $s(xorr(e_1))$) is the and (resp. or, xor) of all the bits in the bit vector $s(e_1)$,

- $te(neg(e_1)) = \texttt{SInt}<n_1+1>$, $te(not(e_1)) = \texttt{UInt}<n_1>$, moreover,
  - $s(neg(e_1)) = bv_{\texttt{SInt}<n_1+1>(-s(e_1))}$,
  - $s(not(e_1))$ is obtained by applying the logical not to each bit of the bit vector $s(e_1)$,
- $te(cvt(e_1)) = \texttt{SInt}<n_1+1>$ if $\theta = \texttt{UInt}$, and $te(cvt(e_1)) = \texttt{SInt}<n_1>$ otherwise, moreover,
  - if $\theta = \texttt{UInt}$, then $s(cvt(e_1)) = bv_{\texttt{SInt}<n_1+1>(s(e_1))}$,
  - if $\theta = \texttt{SInt}$, then $s(cvt(e_1)) = s(e_1)$,

- let $te(e_1) = \theta<n_1>$, where $\theta \in \{\texttt{UInt}, \texttt{SInt}\}$, then
  - $te(pad(e_1, n)) = \theta<\max(n_1, n)>$, moreover, if $n_1 < n$, then $s(pad(e_1, n)) = bv_{\theta<n>(s(e_1))}$, otherwise, $s(pad(e_1, n)) = s(e_1)$,
  - $te(shl(e_1, n)) = \theta<n_1+n>$, moreover, $s(shl(e_1, n)) = bv_{\theta<n_1+n>(2^n \times s(e_1))}$,
  - $te(shr(e_1, n)) = \theta<n_1-n>$ if $n_1 > n$, and $te(shr(e_1, n)) = \theta<1>$ otherwise, moreover,
    - if $n_1 > n$, then $s(shr(e_1, n))$ is obtained from $s(e_1)$ by shifting n bits right,
    - otherwise, if $\theta = \texttt{UInt}$, then $s(shr(e_1, n)) = bv_{\texttt{UInt}<1>(0)}$, otherwise, $s(shr(e_1, n)) = bv_{\texttt{SInt}<1>(0)}$ if $s(e_1) \geq 0$, and $s(shr(e_1, n)) = bv_{\texttt{SInt}<1>(1)}$ if $s(e_1) < 0$,
  - $te(head(e_1, n)) = \texttt{UInt}n$, $te(tail(e_1, n)) = \texttt{UInt}n_1 - n$, moreover, $s(head(e_1, n))$ is the most significant n bits of $s(e_1)$, $s(tail(e_1, n))$ is obtained from $s(e_1)$ by truncating the n most significant bits of $s(e_1)$,
  - $te(bits(e_1, n, n')) = \texttt{UInt}<n'-n+1>$, and $s(bits(e_1, n, n'))$ is the bit vector comprising the $n, n+1, \cdots, n'$-th bit of $s(e_1)$.

B MISSING DETAILS IN SECTION 4

The definition of `type_of_expr`(e) . We define the type of HiFIRRTL program in the Coq as $ftype$. In general, it can be used to represent either a *ground type* ($gtyp$), a *vector type* ($vtyp$), or a `bundle type` ($btyp$),

$$ftype ::= gtyp \mid vtyp\ (ftype\ n) \mid btyp\ fields$$

where

$$fields ::= fnil \mid field\ fid\ flip\ ftype\ fields.$$

For each kind of expression of HiFIRRRL, we derived its type t in $ftype$ as follow.

- `type_of_expr(id)` $= t$, where reference id is the entry in the component environment ce.
- `type_of_expr(id.fid)` requires to iteratively match the type of id to find the same fid and then to return the $ftype$ associated. If the field identifier fid can not be found in the $btyp$ of the reference id or the type of id is not $btyp$, it returns $\perp$.
- `type_of_expr(id[n])` returns the base $ftype$ of reference id. If the nature number $n = 0$ or the type of reference id is other than a $vtyp$, it returns $\perp$.
- let `type_of_expr`$(e_1) = t_1$ and `type_of_expr`$(e_2) = t_2$, then `type_of_expr`$(\texttt{mux}(e_0, e_1, e_2)) =$ `max_width`(e_1, e_2), where the `max_width` compute recursively the max width of the (sub) components, iff t_1 t_2 are type equivalent, otherwise it returns $\perp$.
- `type_of_expr`$(\texttt{validif}(e_1, e_2))$ = `type_of_expr`(e_2), regardless the type of $e1$.
- `type_of_expr`(op), op includes all the binary and unary operations, in which only $gtyp$ operation will involve. The `type_of_expr` computes the type as same as te.

C DETAILED RESULTS OF SECTION 3.3

Table 3. Results of conformance testing on the benchmarks from Treadle

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
AggregatePassThrough	72	13.83	3.57	2.87	✓
ALU	61	24.65	2.58	8.56	✓
BundlePassThrough	27	8.71	2.16	3.04	✓
ConstantTapTransposedStreamingFIR	35	11.48	1.40	7.19	✓
DoesAbs	36	11.67	2.89	3.04	✓
DriverTest	8	6.43	0.63	9.14	✓
EqModule	9	8.29	0.64	12.05	✓
ExampleModule	44	11.47	2.00	4.72	✓
FixedPointAdder	11	9.36	0.47	18.89	✓
FixedPointDivide	10	7.30	0.40	17.22	✓
FixedPointReduce	43	18.40	2.38	6.72	✓
FixedPointShifter	15	10.11	1.59	5.36	✓
FixedPrecisionChanger	10	9.51	0.64	13.83	✓
FixedRing1	26	9.26	2.81	2.30	✓
GCD	30	9.75	2.11	3.61	✓
HardwareHashMaker	122	28.38	31.49	-0.10	✓
IntegerModule	11	7.35	0.74	8.88	✓
OverflowTypeCircuit	34	9.90	2.16	3.57	✓
ParameterizedNumberOperation	12	10.93	0.52	19.89	✓
PeekPokeBundleSpecMyCircuit	17	9.11	2.01	3.53	✓
RealGCD2	44	9.98	2.60	2.84	✓
RealGCD3	48	12.66	2.71	3.67	✓
RemoveMantissa	10	8.48	0.55	14.39	✓
RingModule	11	6.91	0.64	9.72	✓
ShouldBeBadUIntSubtractWithGrow	14	7.81	0.66	10.93	✓
SignedAdder	11	9.77	0.68	13.47	✓
SignedModule	34	6.79	1.36	4.01	✓
SimpleAdder	15	9.14	0.69	12.19	✓
SimpleCaseClassModule	10	8.78	0.52	15.87	✓
SimpleComplexAdder	40	12.68	1.56	7.11	✓
Sort	85	17.53	7.14	1.46	✓
UseMyBundle	59	8.75	3.11	1.82	✓
VecPassThrough	75	20.21	5.06	3.00	✓
VF	64	6.53	2.18	2.00	✓
ContextNestingTop	64	8.76	3.10	1.83	✓
MultiGcdCalculator	133	51.35	18.33	1.80	✓
StreamingAutocorrelator	59	8.78	1.56	4.62	✓
Average	**38.08**	**12.18**	**3.13**	**7.16**	

Table 4. Results of conformance testing on the benchmarks from [?]

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
Accumulator	13	17.47	0.67	25.15	✓
Combinational	12	16.70	0.66	24.15	✓
FibCircuit	14	19.02	0.46	40.80	✓
ShiftRegister	21	19.01	0.63	29.16	✓
Average	**15.00**	**18.05**	**0.60**	**29.81**	

Table 6. Results of conformance testing on the benchmarks from the Chisel Book

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
Adder	11	65.28	6.31	9.35	✓
Alu	24	70.28	10.96	5.41	✓

Continued on next page

Table 6 – continued from previous page

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
Arbiter3	26	62.02	11.91	4.21	✓
Arbiter3Direct	25	72.30	10.46	5.91	✓
Arbiter3Loop	29	62.55	11.62	4.38	✓
ArbiterSimpleTree	230	317.82	84.72	2.75	✓
ArbiterTree	266	327.34	99.69	2.28	✓
BasicExercise	11	90.55	6.74	12.44	✓
BasicPrefixAdder	384	181.39	123.58	0.47	✓
BcdTable	210	573.42	78.34	6.32	✓
BundleVec	405	210.73	121.84	0.73	✗
CarryLookaheadAdder	303	295.27	101.40	1.91	✓
CarryRippleAdder	51	83.15	24.64	2.37	✓
CarrySelectAdder	151	204.53	58.35	2.51	✓
CarrySkipAdder	135	569.89	46.82	11.17	✓
CombElseWhen	12	90.20	6.47	12.95	✓
Combinational	15	105.54	62.88	0.68	✓
CombOther	10	60.97	6.80	7.96	✓
CombWhen	10	64.73	6.44	9.06	✓
CombWhen2Untested	11	63.13	5.50	10.49	✓
CombWireDefault	10	64.79	6.36	9.20	✓
Comparator	13	80.62	11.11	6.26	✓
CompFn	28	113.24	18.37	5.16	✓
Conditional	16	59.62	7.01	7.51	✓
Count100	14	66.17	7.48	7.85	✓
CountBool	32	105.17	11.23	8.37	✓
CounterDevice	59	126.75	26.96	3.70	✓
Debounce	53	91.03	16.12	4.65	✓
DownCounter	17	90.88	12.92	6.03	✓
DownTicker	15	69.62	7.56	8.21	✓
DrawAcc	24	82.80	10.32	7.02	✓
DrawMux6	21	71.95	10.17	6.08	✓
EncDec	102	179.80	19.76	8.10	✓
FifoRegister	32	96.69	21.14	3.57	✓
FileReader	521	585.59	175.67	2.33	✓
Flasher2	65	89.12	26.32	2.39	✓
FunctionalAdd	20	76.17	10.52	6.24	✓
FunctionalComp	15	73.99	11.12	5.66	✓
FunctionalMin	159	685.49	72.80	8.42	✓
FunctionCounter	31	82.91	16.06	4.16	✓
GenHardware	60	106.13	60.44	0.76	✓
Hello	19	71.31	6.24	10.43	✓
Logic	50	102.55	30.10	2.41	✓
MemMappedRV	42	131.11	29.59	3.43	✓
MinFanoutPrefixAdder	309	306.08	101.16	2.03	✓
MuxCounter	17	92.77	11.40	7.14	✓
NerdCounter	19	78.46	13.30	4.90	✓
NerdTicker	16	64.69	7.75	7.35	✓
ParamAdder	11	67.05	6.68	9.03	✓
ParamFunc	23	69.88	16.31	3.29	✓
Pwm	49	542.35	23.61	21.98	✓
ReadyValidBuffer	27	94.96	17.75	4.35	✓
Register	11	73.70	5.17	13.25	✓
Registers	17	75.07	5.43	12.81	✓
RisingFsm	24	89.53	10.71	7.36	✓
RisingMooreFsm	28	74.30	12.24	5.07	✓
ScalaGen	39	69.43	11.27	5.16	✓
SequCounter	38	101.78	25.86	2.94	✓
Sequential	56	116.07	40.66	1.85	✓
ShiftRegister	33	114.76	19.16	4.99	✓
SimpleFsm	28	102.20	9.23	10.07	✓
SinkClock	32	89.70	12.50	6.18	✓
SourceClock	29	93.15	15.53	5.00	✓
Timer	19	86.62	8.89	8.74	✓
UIntVsU	17	71.30	28.07	1.54	✓
UpTicker	14	67.90	7.29	8.32	✓

Continued on next page

Table 6 – continued from previous page

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
WhenCounter	17	73.21	12.41	4.90	✓
BubbleFifo	108	264.40	122.41	1.16	✓
ClockDomains	76	87.42	36.57	1.39	✓
Count10	40	77.75	12.16	5.39	✓
Formal	24	68.69	10.33	5.65	✓
PopulationCount	98	198.58	59.83	2.32	✓
SyncReset	33	75.61	14.54	4.20	✓
UseAdder	42	73.75	13.60	4.42	✓
UseMemMappedRV	175	251.77	137.40	0.83	✓
UseParamRouter	46	86.47	32.72	1.64	✓
UseParamRouter2	50	77.73	38.20	1.03	✓
Average	**68.86**	**136.90**	**29.73**	**5.81**	

Table 5. Results of conformance testing on our own benchmarks

Benchmark name	#LOC	Execution time (μs)		Speedup	Result
		Treadle	Ours		
Mytest	52	249.90	103.69	1.41	✓
Mytestinst	49	127.70	21.34	4.98	✓
Average	**50.50**	**188.80**	**62.52**	**3.20**	